



PELO FUTURO DO TRABALHO

Métodos de ordenação

Profº Gustavo Garcia de Amo

Por que ordenar

A ordenação ou classificação de registros consiste em organizá-los em ordem crescente ou decrescente e assim facilitar a recuperação desses dados. A ordenação tem como objetivo facilitar as buscas e pesquisas de ocorrências de determinado elemento em um conjunto ordenado.

Na computação existe uma série de algoritmos que utilizam diferentes técnicas de ordenação para organizar um conjunto de dados, eles são conhecidos como Métodos de Ordenação ou Algoritmos de Ordenação. Vamos conhecer um pouco mais sobre eles.

Ordenação

Na escolha de um algoritmo de ordenação interna deve ser considerado principalmente:

- O tempo gasto pela ordenação;
- O uso econômico da memória disponível;

A eficiência no manuseio de dados muitas vezes pode ser aumentada se os dados estiverem dispostos de acordo com algum critério de ordem

- Exemplo: Uma lista telefônica sem ordem.

Tipos de ordenação

Ordenação Interna: onde todos os elementos a serem ordenados cabem na memória principal e qualquer registro pode ser imediatamente acessado.

Ordenação Externa: onde os elementos a serem ordenados não cabem na memória principal e os registros são acessados sequencialmente ou em grandes blocos.

Métodos Simples

Métodos Simples

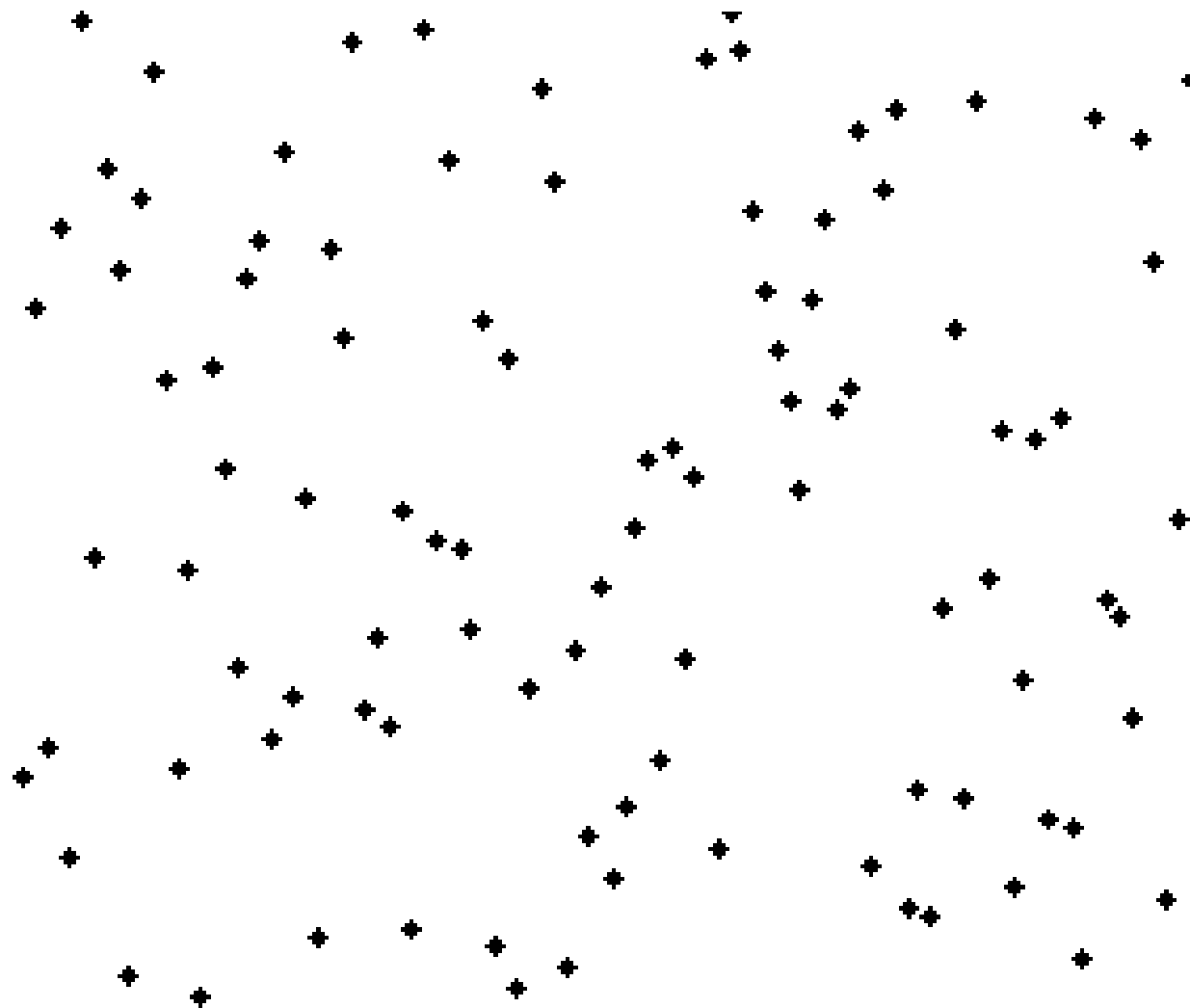
Os métodos simples são adequados para pequenos vetores, são programas pequenos e fáceis de entender. Exemplos: *Insertion Sort*, *Selection Sort*, *Bubble Sort*, *Comb Sort*.

Insertion Sort

Insertion Sort ou ordenação por inserção é o método que percorre um vetor de elementos da esquerda para a direita e à medida que avança vai ordenando os elementos à esquerda.

O funcionamento do algoritmo é bem simples: consiste em cada passo a partir do segundo elemento selecionar o próximo item da sequência e colocá-lo no local apropriado de acordo com o critério de ordenação.

6 5 3 1 8 7 2 4



Insertion Sort



Insertion Sort

Explicar no algoritmo

Atividade

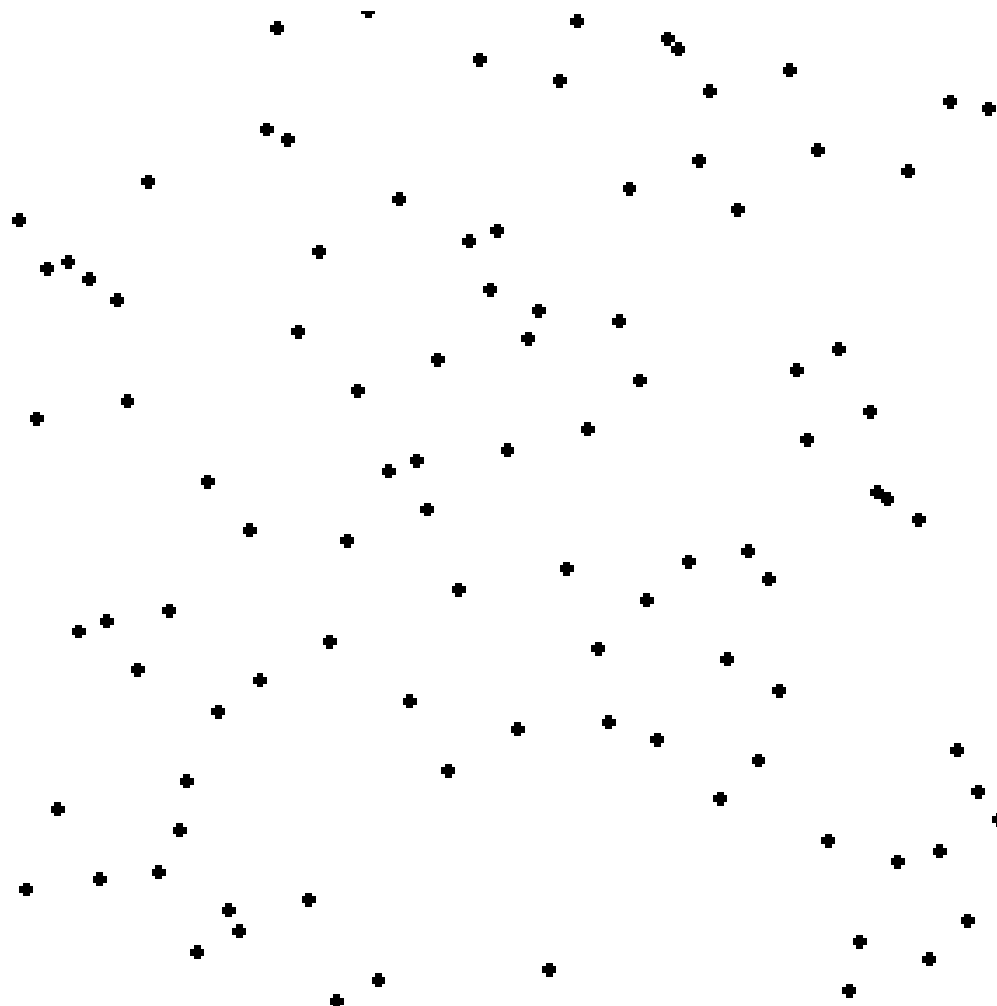
- 1- Desenvolva um fluxograma da ordenação insertion Sort.
- 2 Insira 8 valores em um vetor e aplique o insertion Sort para organizar o vetor. Escreva o vetor original e o organizado
- 3: insira 5 valores em um vetor e aplique o insertion Sort para organizar o vetor de forma decrescente.
- 4: insira 50 valores em um vetor (sorteio) e utilizando o insertion Sort em uma função (passe o vetor como parâmetro), organize-o em forma crescente e decrescente. Exiba o vetor original, crescente e decrescente.

Selection Sort

A ordenação por seleção ou *selection sort* consiste em selecionar o menor item e colocar na primeira posição, selecionar o segundo menor item e colocar na segunda posição, segue estes passos até que reste um único elemento.

Selection Sort

	8
	5
	2
	6
	9
	3
	1
	4
	0
	7



Selection Sort



Selection Sort

Explicar no algoritmo

Atividade

0- Desenhar o fluxograma da estrutura de selection Sort

1- Ordenação Crescente: Crie um programa que leia um vetor com 10 números inteiros e utilize o algoritmo de Selection Sort para ordenar o vetor em ordem crescente. Em seguida, exiba o vetor ordenado.

2- Ordenação Decrescente: Crie um programa que leia um vetor com 8 números reais e utilize o algoritmo de Selection Sort para ordenar o vetor em ordem decrescente. Em seguida, exiba o vetor ordenado.

3- Maior e Menor Elemento: Crie um programa que leia um vetor com 12 números inteiros e utilize o algoritmo de Selection Sort para encontrar e exibir o maior e o menor elemento presente no vetor.

Atividade

4- Números Pares e Ímpares: Crie um programa que leia um vetor com 15 números inteiros e utilize o algoritmo de Selection Sort para separar os números pares dos números ímpares. Em seguida, exiba os números pares e os números ímpares em vetores separados.

5- Média dos Elementos: Crie um programa que leia um vetor com 9 números reais e utilize o algoritmo de Selection Sort para ordenar o vetor em ordem crescente. Em seguida, calcule e exiba a média dos elementos do vetor.

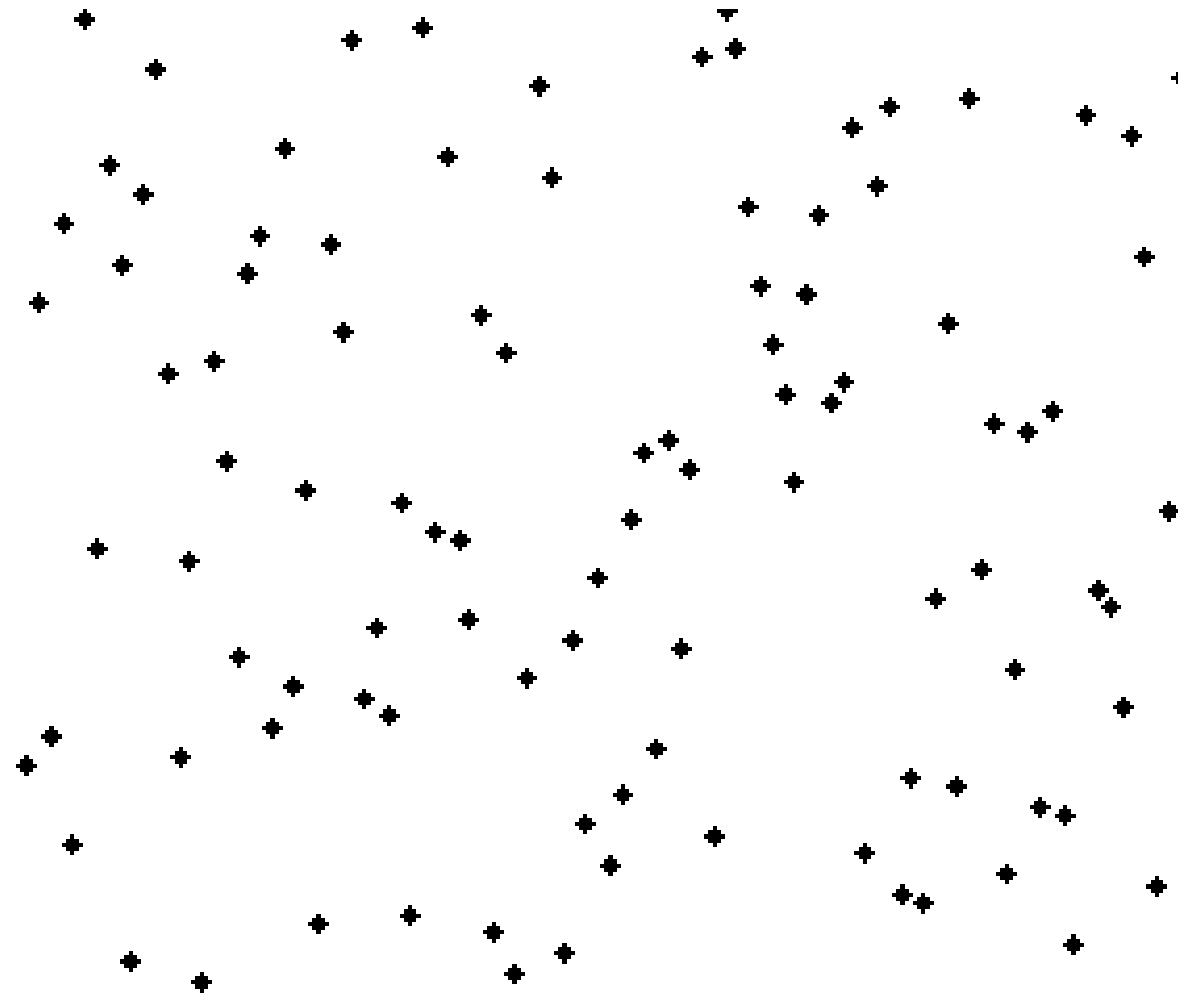
6- Inversão de Vetor: Crie um programa que leia um vetor com 10 números inteiros e utilize o algoritmo de Selection Sort para ordenar o vetor em ordem crescente. Depois, crie um novo vetor com os elementos do vetor original em ordem inversa e exiba-o.

Bubble Sort

Bubble Sort é um algoritmo de ordenação que pode ser aplicado em Arrays e Listas dinâmicas. Se o objetivo é ordenar os valores em forma decrescente, então, a posição atual é comparada com a próxima posição e, se a posição atual for maior que a posição posterior, é realizada a troca dos valores nessa posição. Caso contrário, não é realizada a troca, apenas passa-se para o próximo par de comparações.

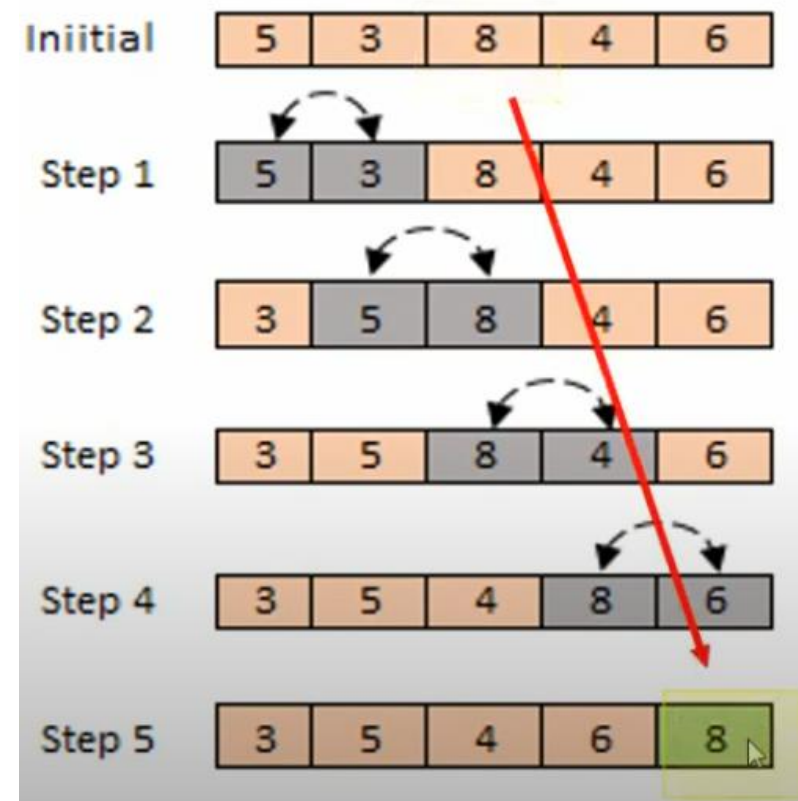
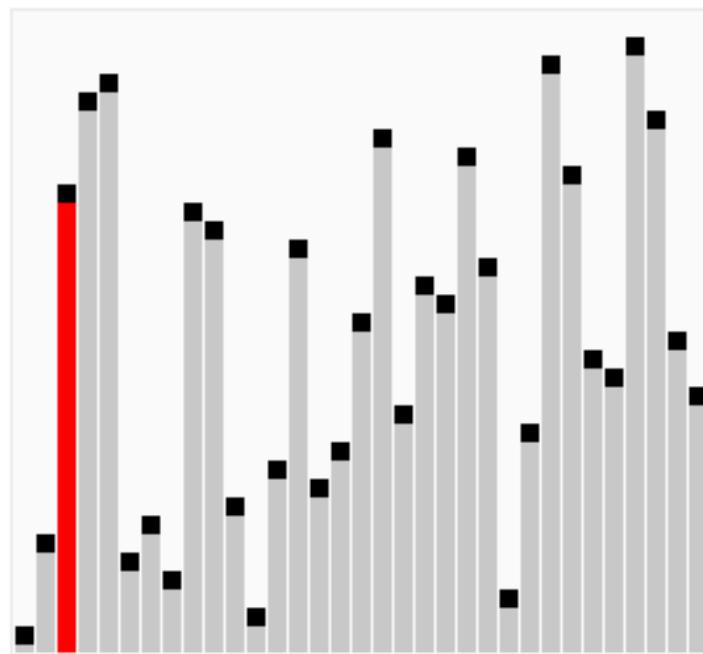
Se o objetivo é ordenar os valores em forma crescente, então, a posição atual é comparada com a próxima posição e, se a posição atual for menor que a posição posterior, é realizada a troca. Caso contrário, a troca não é feita e passa-se para o próximo par de comparação.

Bubble Sort



Bubble Sort

- Realiza ordenação por troca
- Percorre o vetor várias vezes, comparando um elemento com o seu sucessor



Bubble Sort

- Explicar no algoritmo

Atividade bubble Sort

1- Ordenar um vetor de 15 posições em ordem crescente e decrescente. Exibir as duas ordens e o vetor original

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

2- Ordene os números 1,5,6,9,7,10,598,218,55

3-Ordene os números 58,15,2,0,1,8,30,100

Comb Sort

O algoritmo de ordenação Comb Sort é uma variação do algoritmo Bubble Sort que busca melhorar o desempenho em casos de ordenação de vetores com elementos distantes de suas posições finais.

O funcionamento do Comb Sort é baseado em uma ideia simples, porém eficaz. Ele utiliza um parâmetro chamado de "gap" (lacuna) que representa a distância entre os elementos que serão comparados e trocados durante o processo de ordenação.

Comb Sort - GAP

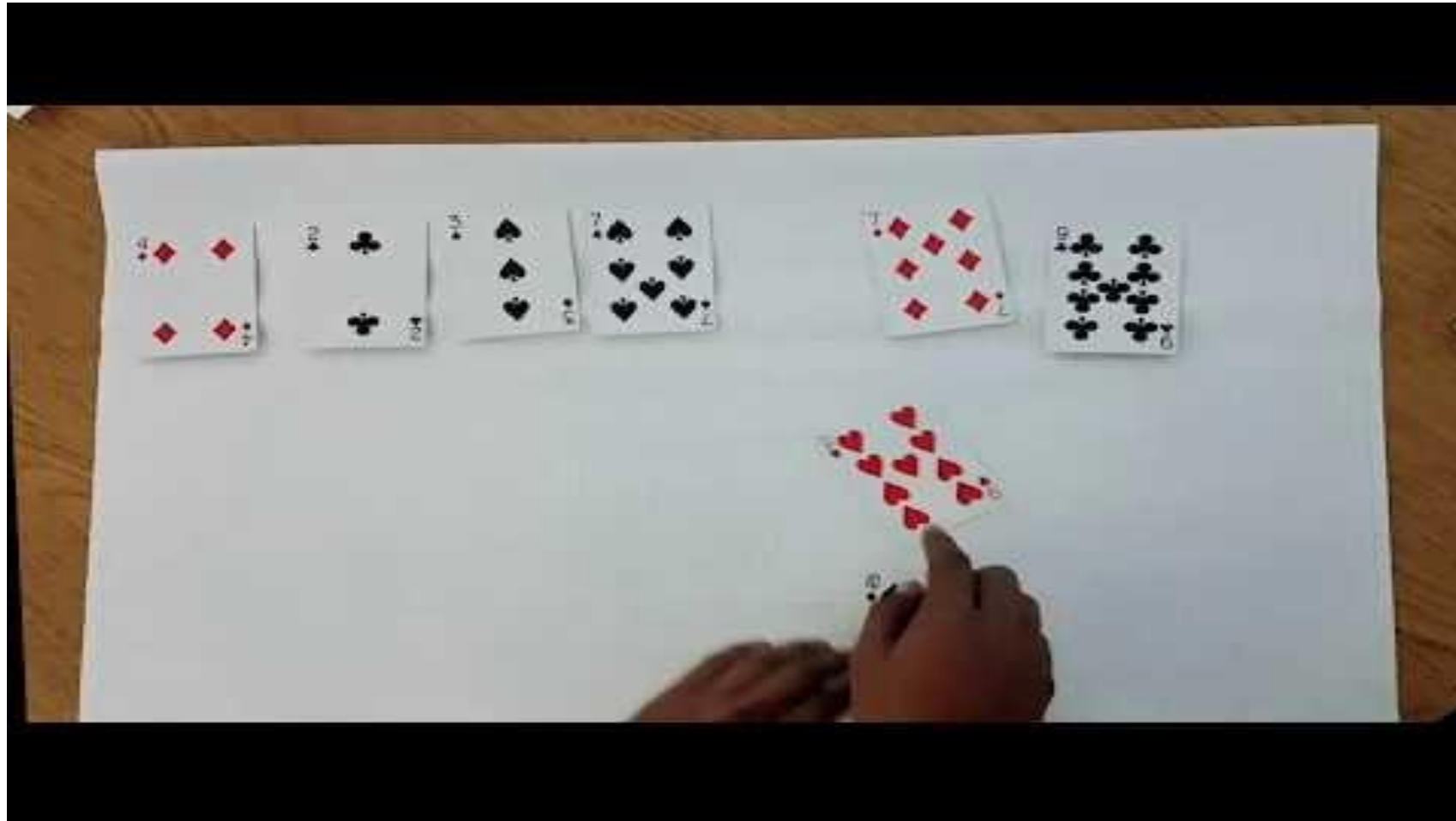
Inicialmente, o valor do gap é definido como o tamanho total do vetor a ser ordenado. Em seguida, um fator de redução é aplicado ao gap em cada iteração, geralmente 1.3, embora outros valores também possam ser utilizados. Isso significa que o gap é reduzido em 30% a cada iteração.

O algoritmo então percorre o vetor comparando os elementos que estão a uma distância de gap um do outro. Se o elemento da posição atual for maior do que o elemento que está gap posições à frente, ocorre uma troca entre eles. Esse processo é repetido enquanto o gap for maior ou igual a 1 e houver pelo menos uma troca realizada durante a iteração anterior.

Comb Sort - GAP

Após percorrer todo o vetor com o gap atual, o valor do gap é atualizado e o processo é repetido. Essa etapa é realizada até que o gap seja igual a 1 e não ocorram mais trocas durante a iteração.

Comb Sort



Comb Sort

- Explicar no algoritmo

Atividades

Ordenação Decrescente: Faça um programa que leia uma lista de nomes de alunos e suas respectivas notas, e então ordene a lista em ordem decrescente de acordo com as notas. Usar o Comb Sort.

Ordenação Crescente: Escreva um programa que solicite ao usuário que insira uma lista de 10 números e, em seguida, ordene essa lista em ordem crescente. Você pode utilizar qualquer método de ordenação, como Bubble Sort, Selection Sort ou Insertion Sort

Escreva um programa que leia o nome e idade de 10 pessoas. Posterior, o usuário deverá informar qual método de ordenação ele irá utilizar para ordenar as idades em formato decrescente. (métodos que o usuário poderá escolher são: Selection, Insertion, Bubble ou Comb Sort). Utilize uma função pra cada método de ordenação

Métodos Eficientes

Métodos Eficientes

Os métodos eficientes são mais complexos nos detalhes, requerem um número menor de comparações. São projetados para trabalhar com uma quantidade maior de dados e possuem complexidade. Exemplos: Quick sort, Merge sort, Shell sort, Heap sort, Radix sort, Gnome sort, Count sort, Bucket sort, Cocktail sort, Timsort.

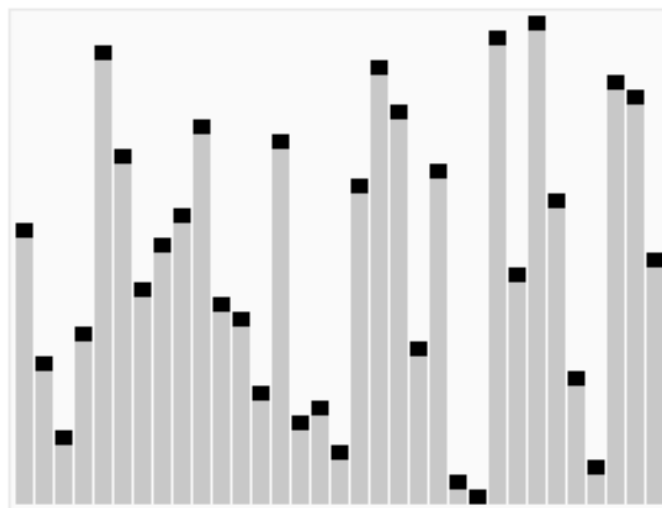
Quick Sort

É um algoritmo de comparação que emprega a estratégia de “divisão e conquista”. A ideia básica é dividir o problema de ordenar um conjunto com n itens em dois problemas menores. Os problemas menores são ordenados independentemente e os resultados são combinados para produzir a solução final.

Basicamente a operação do algoritmo pode ser resumida na seguinte estratégia: divide sua lista de entrada em duas sub-listas a partir de um pivô, para em seguida realizar o mesmo procedimento nas duas listas menores até uma lista unitária.

Quick Sort - Funcionamento

- Escolhe um elemento da lista chamado pivô.
- Reorganiza a lista de forma que os elementos menores que o pivô fiquem de um lado, e os maiores fiquem de outro. Esta operação é chamada de “particionamento”.
- Recursivamente ordena a sub-lista abaixo e acima do pivô.



Quick Sort - Funcionamento



Quick Sort

- Explicar no algoritmo

Atividade

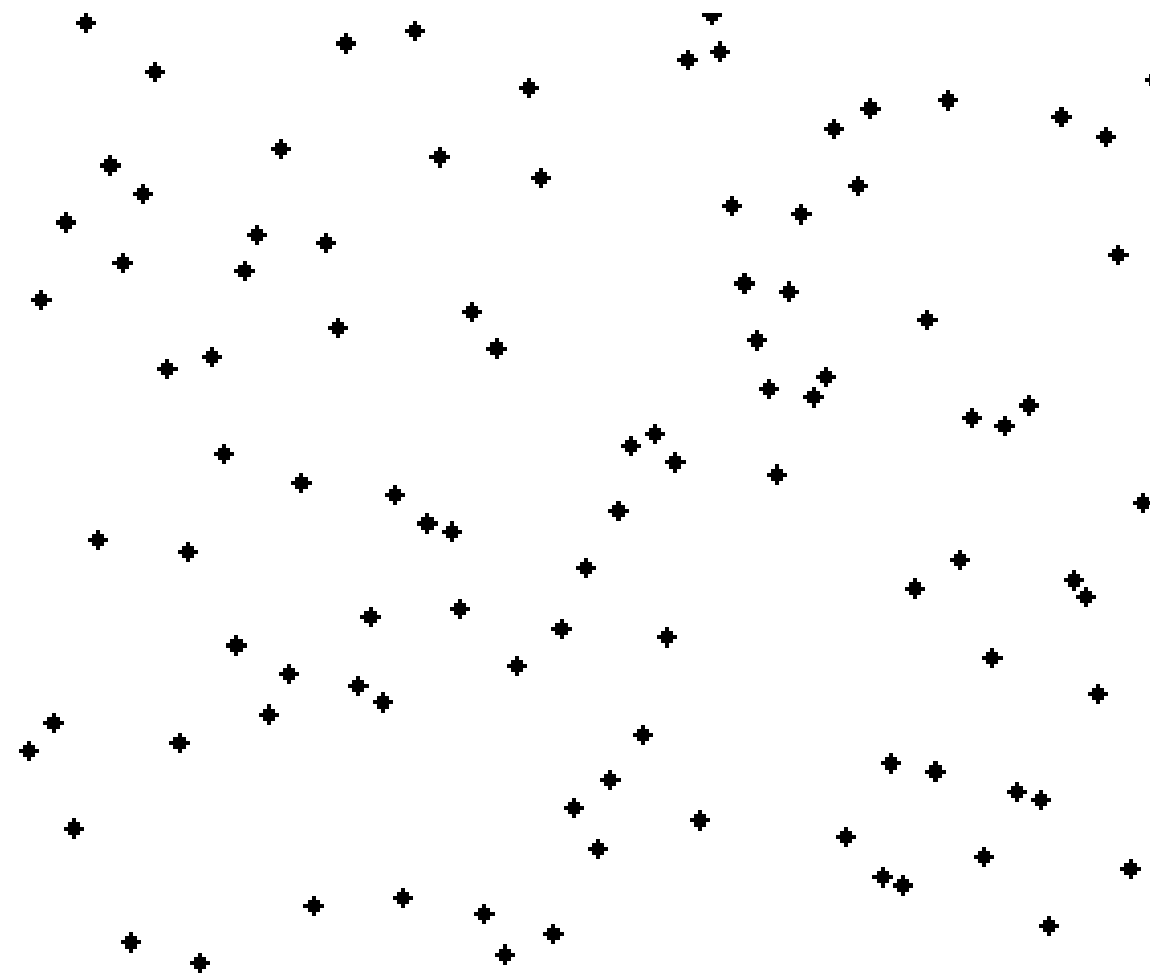
- Organize um vetor de 7 números utilizando o método de ordenação Quick sort
- insira 5 valores em um vetor e aplique o Quick sort para organizar o vetor de forma decrescente.
- insira 50 valores em um vetor (sorteio) e utilizando o Quick sort em uma função (passe o vetor como parâmetro), organize-o em forma crescente e decrescente. Exiba o vetor original, crescente e decrescente.

Merge Sort

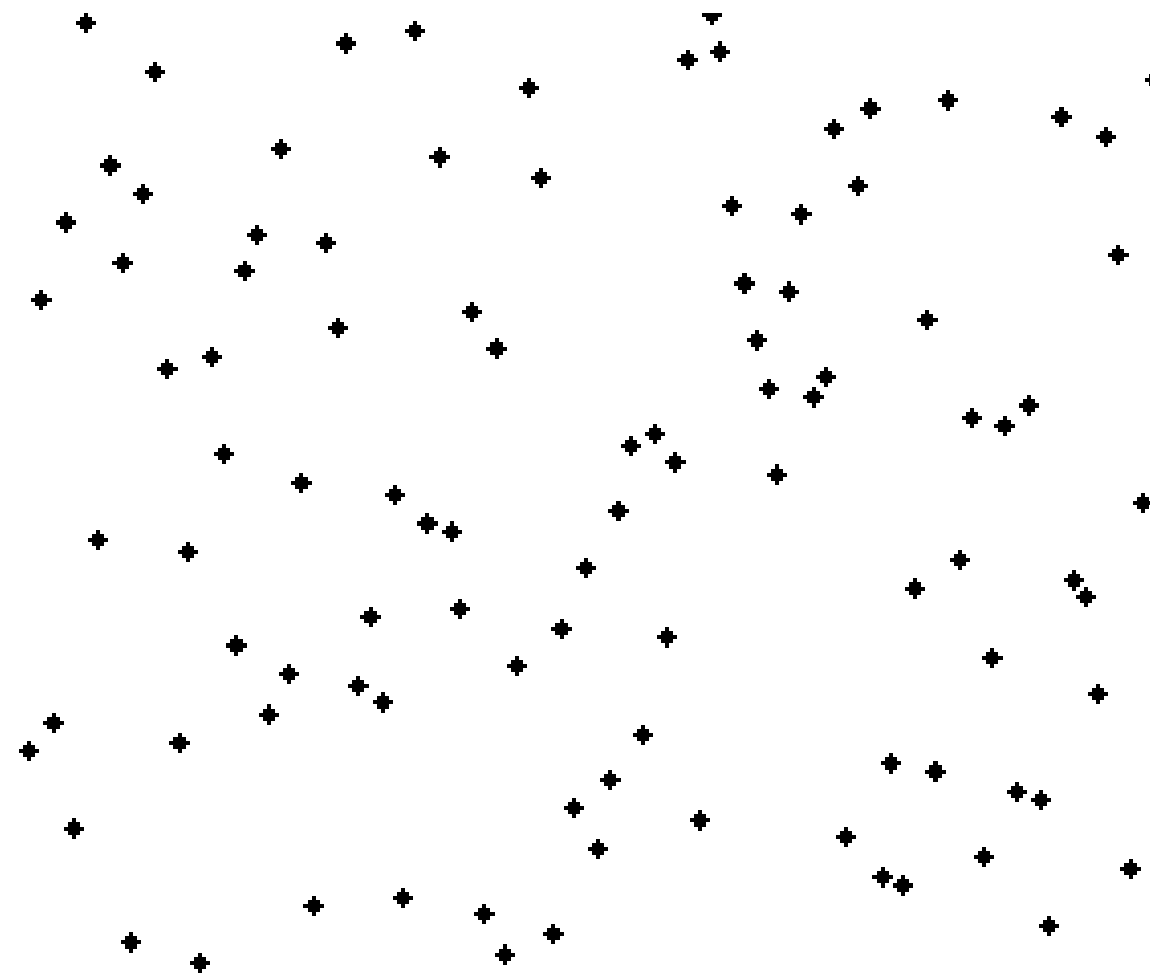
Esse algoritmo divide o problema em pedaços menores, resolve cada pedaço e depois junta (merge) os resultados. O vetor será dividido em duas partes iguais, que serão cada uma divididas em duas partes, e assim até ficar um ou dois elementos cuja ordenação é trivial.

Para juntar as partes ordenadas os dois elementos de cada parte são separados e o menor deles é selecionado e retirado de sua parte. Em seguida os menores entre os restantes são comparados e assim se prossegue até juntar as partes.

Merge Sort



Merge Sort



Merge Sort



Merge Sort

- Aplicar o Merge Sort no algoritmo 40 15 16 40 46 49 22
- Utilizando uma função com o Merge Sort implementado. Organize os números: 80 50 10 30 550 111 112 231
- Implemente uma função com o Merge sort para organizar 10 números informados pelo usuário.
- Implemente uma função com o Merge sort para organizar 5 números informados pelo usuário. Converta-os para binário e exiba-os.

Materiais de Apoio

<https://www.treinaweb.com.br/blog/conheca-os-principais-algoritmos-de-ordenacao>

<https://embarcados.com.br/algoritmos-de-ordenacao-bubble-sort/>



PELO FUTURO DO TRABALHO

sesisc.org.br     **sc.senai.br**

0800 048 1212

Rodovia Admar Gonzaga, 2765 - Itacorubi - 88034-001 - Florianópolis, SC